

MEMORIA TÉCNICA DEL PROYECTO FINAL: GESTIÓN DE TIENDA DE ROPA ARTESANAL



Autor: Jaime Martín Benítez

Curso: 1º de Desarrollo de Aplicaciones Multiplataforma (DAM)

Fecha: 26 Mayo de 2026

1. INTRODUCCIÓN

El presente proyecto consiste en el diseño e implementación de una aplicación informática orientada a la gestión integral de una tienda de ropa artesanal. El objetivo principal es ofrecer una solución robusta y eficiente para el control de inventario polimórfico, el registro único de clientes, la gestión interactiva de carritos de compra (pedidos) y la explotación de datos mediante informes avanzados.

La aplicación se ha desarrollado íntegramente en Java, aplicando los paradigmas de la Programación Orientada a Objetos (POO), persistencia en ficheros planos de texto (CSV), gestión avanzada de excepciones propias y procesamiento de flujos de datos mediante la Stream API. Para garantizar la usabilidad, toda la interacción se realiza mediante una interfaz de consola parametrizada y blindada contra entradas erróneas del usuario.

2. ARQUITECTURA DEL SISTEMA Y DISEÑO ORIENTADO A OBJETOS

La arquitectura del software se fundamenta en un diseño modular y fuertemente cohesionado, dividido en tres capas principales: modelo de datos (tienda.modelo), lógica de negocio o servicios (tienda.servicio) e interfaz de usuario (tienda.ui).

Jerarquía de Herencia y Polimorfismo

El núcleo del modelo se estructura en torno a la clase abstracta e idónea Ropa. De ella emanan tres subclases abstractas intermedias especializadas que clasifican el catálogo textil:

- **PrendaSuperior:** Modela prendas con el atributo específico tipoManga. De ella heredan las clases concretas Camiseta, Sudadera y Body.
- **PrendaInferior:** Modela prendas con el atributo largo. De ella heredan las clases concretas Cargo y Vaquero y Falta.

- **PrendaCuerpoCompleto:** Modela piezas de una sola estructura articuladas mediante el atributo largo. De ella deriva la clase concreta Vestido.

El uso de métodos abstractos (como toString()) obliga a las clases hijas a implementar su propio comportamiento descriptivo, permitiendo al sistema tratar de forma homogénea a cualquier artículo del inventario a través del polimorfismo dinámico.

Diseño de Interfaces Funcionales y Segregación

Para dotar de comportamientos específicos y adicionales a ciertas prendas sin sobrecargar la clase base ni violar los principios SOLID, se han implementado tres interfaces:

- **Personalizable:** Implementada por Camiseta. Permite añadir opciones de personalización artesanal (Estampado o Bordado) calculando un coste extra dinámico.
- **CuidadoEspecial:** Implementada por Body, Falda y Vestido. Exige la definición de instrucciones detalladas de lavado y conservación según la delicadeza de los tejidos artesanales.
- **EmpaquetablePremium:** Implementada por Sudadera y Vestido. Permite activar envoltorios especiales de regalo y deducir suplementos económicos si se incorpora una tarjeta con mensaje dedicado.

3. ESTRUCTURAS DE DATOS Y JUSTIFICACIÓN TÉCNICA

La elección de las colecciones del framework Java Collections integradas en la clase GestorRopa responde estrictamente a criterios de rendimiento, optimización de memoria y lógica de negocio:

- **HashMap<String, Ropa> inventario:** Se ha elegido un mapa asociativo donde la clave es el identificador único del artículo (idArticulo normalizado en mayúsculas) y el valor es el objeto Ropa. Esta estructura garantiza búsquedas, inserciones y eliminaciones directas, evitando tener que recorrer secuencialmente el catálogo completo cada vez que un cliente compra una prenda en el carrito.

- **HashSet<Cliente> clientes:** Se ha seleccionado un conjunto para almacenar los clientes de la tienda. Dado que el DNI es un identificador único, el HashSet (en combinación con la sobrescritura de los métodos hashCode() y equals() en la clase Cliente) asegura de manera nativa e interna que no puedan existir dos clientes con el mismo DNI en el sistema, protegiendo la integridad del registro.
- **ArrayList<Pedido> historialPedidos:** El historial de ventas requiere almacenar los pedidos respetando de forma estricta su orden cronológico de inserción y permitiendo el acceso rápido por índice correlativo. La lista indexada ArrayList es óptima para este propósito de auditoría y almacenamiento secuencial.
- **Repositorio<T>:** Como muestra de diseño avanzado y abstracción, se ha creado una clase genérica parametrizada <T> que actúa como contenedor universal. Esta clase desacopla la colección interna de las entidades de negocio, permitiendo reutilizar operaciones de inserción, borrado y filtrado predictivo para cualquier objeto futuro de la aplicación, sentando las bases del patrón arquitectónico DAO (Data Access Object).

4. PERSISTENCIA EN FICHEROS (E/S)

La persistencia se gestiona de forma local y secuencial mediante archivos de texto plano con formato de valores separados por punto y coma (CSV), localizados en un directorio externo denominado recursos/. Las tres entidades cuentan con rutas dedicadas: inventario.csv, clientes.csv y historial_pedidos.csv.

Mecanismo de Serialización y Delegación

Para cumplir con los principios de encapsulamiento y responsabilidad única, la lógica de formateo no reside en el gestor, sino en cada entidad a través del método getFichaArchivo().

- La clase Ropa y sus descendientes vuelcan sus campos delimitados por la constante estática Ropa.SEPARADOR (";"), incluyendo al principio el nombre abreviado de su clase (this.getClass().getSimpleName()) para permitir al gestor discriminar el tipo de objeto exacto durante la lectura.

- La clase Pedido genera cadenas complejas donde los artículos del carrito se serializan concatenando sus códigos únicos separados por comas, controlando los casos de pedidos vacíos mediante la etiqueta "VACIO".

Flujos de Lectura y Escritura Robusta

La carga de datos (`cargarDatos()`) se realiza al arrancar el hilo principal mediante `BufferedReader` y `FileReader`. El analizador léxico (`parser`) lee línea a línea, fragmenta las cadenas mediante `.split(";")` y reconstruye los objetos en memoria mediante estructuras `switch-case` polimórficas. El captador controla de forma silenciosa la excepción `FileNotFoundException` para garantizar que, si se trata del primer arranque de la tienda, la aplicación se inicialice con colecciones vacías de forma transparente y sin colgarse.

El volcado masivo (`guardarDatos()`) se ejecuta automáticamente al pulsar la opción de salida del menú principal empleando `BufferedWriter` y `FileWriter`. El gestor comprueba previamente la existencia del directorio físico de recursos mediante la clase `File` y, si no existe en la raíz del disco duro, invoca al método `.mkdir()` para crearlo de forma segura antes de la escritura.

5. FUNCIONALIDADES AVANZADAS: EXCEPCIONES Y STREAM API

Gestión de Excepciones Propias

El software rechaza el uso de códigos de error numéricos o impresiones directas de texto en la lógica, empleando en su lugar un sistema estructurado de excepciones controladas (`checked exceptions`) que heredan de `Exception`:

StockAgotadoException: Lanzada por el método `verificarDisponibilidad(cantidad)` en `Ropa` si el número de unidades solicitadas en un pedido supera las existencias físicas del almacén, interrumpiendo el flujo del carrito de inmediato.

ArticuloNoEncontradoException: Lanzada por el GestorRopa al intentar buscar o dar de baja una prenda cuyo ID no existe en el HashMap, o al intentar retirar un artículo del carrito que nunca fue añadido.

EstadoPedidoInvalidoException: Protege el ciclo de vida de las ventas en la clase Pedido. Impide transiciones ilegales de estado, arrojando un error crítico si el usuario intenta cancelar un pedido que ya consta como COMPLETADO o si intenta alterar un pedido previamente CANCELADO.

Explotación de Datos Mediante la Stream API

El potencial analítico del programa se concentra en el submenú de consultas avanzadas, implementado mediante flujos declarativos de Streams y expresiones lambda:

Ingresos Totales de Caja: Mediante `historialPedidos.stream()`, el programa aplica un filtro `.filter(p -> p.getEstado() == EstadoPedido.COMPLETADO)` para aislar únicamente las ventas cobradas. Posteriormente, transforma el flujo de objetos a un tipo primitivo decimal con `.mapToDouble(Pedido::calcularTotal)` (donde el polimorfismo calcula de forma implícita los costes extras de empaquetados o personalizaciones) y acumula el saldo final usando el reductor `.sum()`.

Filtrados y Ordenación: Métodos como `buscarPorTalla` o `buscarPorPrecioMax` procesan el flujo de valores del inventario (`inventario.values().stream()`), aplican predicados ignorando mayúsculas con `.equalsIgnoreCase()` y recolectan los resultados de forma dinámica en nuevas listas utilizando `.collect(Collectors.toCollection(ArrayList::new))`.

Filtrado por Interfaz: El método `getArticulosPersonalizables()` demuestra la potencia de los flujos analizando la herencia en tiempo de ejecución mediante `.filter(r -> r instanceof Personalizable)`, mapeando los objetos a la interfaz correspondiente y devolviendo exclusivamente los artículos que admiten costuras y estampados artesanales.

6. MANUAL DE USUARIO

La aplicación proporciona un entorno de navegación intuitivo a través de la consola de comandos. A continuación se detallan las pautas para operar el sistema:

Menú Principal

Al iniciar la aplicación, se cargan los datos históricos y se despliega el menú raíz con cinco opciones principales:

1. **Gestión de Catálogo (Ropa):** Permite dar de alta cualquiera de las siete prendas del catálogo rellendo sus campos específicos, dar de baja artículos por código identificador, buscar prendas detalladas o imprimir todo el almacén en memoria.
2. **Gestión de Clientes:** Formulario seguro para registrar compradores. El sistema exige y valida que el DNI no esté duplicado antes de solicitar el resto de la información.
3. **Gestión de Pedidos (Ventas):** Abre el módulo del carrito interactivo. Requiere introducir un DNI existente. Una vez validado, se abre un submenú para añadir ropa (lo que disminuye automáticamente el stock general), mostrar el desglose provisional del ticket con sus extras económicos o finalizar el pedido (lo que conmuta su estado a COMPLETADO e ingresa el importe en la caja).
4. **Consultas Avanzadas y Reportes:** Zona de analítica de negocio que ejecuta los flujos de Streams (filtros por talla, presupuestos máximos, prendas personalizables, catálogo ordenado de menor a mayor precio e ingresos totales de la caja registradora).
5. **Guardar y Salir:** Vuelca todas las colecciones a los ficheros CSV correspondientes y cierra la aplicación de forma segura.

7. CONCLUSIONES

El desarrollo de esta plataforma de gestión de ropa artesanal ha permitido consolidar de forma práctica y sinérgica los conocimientos avanzados adquiridos a lo largo del módulo de Programación de 1º de DAM.

La mayor complejidad técnica residió en coordinar la persistencia polimórfica: reconstruir objetos de distintas clases hijas a partir de una única estructura de fichero plano CSV requiere un control exhaustivo de los índices de las cadenas y de las conversiones de tipos de datos. Asimismo, la sincronización de stock en tiempo real al agregar artículos al carrito y el filtrado estricto de los pedidos en estado COMPLETADO para el cálculo de ingresos de caja supusieron retos lógicos que se solventaron con éxito gracias al potencial de la Stream API y al correcto encapsulamiento de los métodos.

En conclusión, el programa resultante constituye una aplicación comercial sólida, escalable, blindada ante excepciones operativas y con una

arquitectura de código limpia que cumple estrictamente con los estándares profesionales del desarrollo de software en Java.